# 实验 6 报告模板

# 6 实验六 传输层的 TCP 协议分析

## 6.1 实验目的

1. 理解 TCP 报文首部格式和字段的作用，TCP 连接的建立和释放过程，TCP 数据传输中的编号与确认（应答）的过程。能够使用工具分析 TCP 的应答机制。

2. 理解 TCP 的错误恢复的工作原理和字节流的传输模式，能够使用工具分析实际的 TCP 流中的错误恢复机制。

3. 理解 TCP 的流量控制的工作原理，能够使用工具分析 TCP 流的流量控制。

4. 理解 TCP 的拥塞控制的工作原理，能够使用工具分析 TCP 流的拥塞控制中的慢启动和快速重传过程。

5. 能够编写程序实现 TCP 的大数据量的传输。

## 6.2 实验内容及原理

1. 使用基于 TCP 的应用程序（比如浏览器下载文件）传输文件，在客户端和服务器均要捕获 TCP 报文。

2. 分析 TCP 报文首部信息、TCP 连接的建立和释放过程、TCP 数据的编号与确认机制。观察几个典型的 TCP 选项：MSS、SACK、Window Scale、Timestamp 等，查资料说明其用途。

3. 观察和估计客户机到服务器的 RTT，双方各自的 MSS。

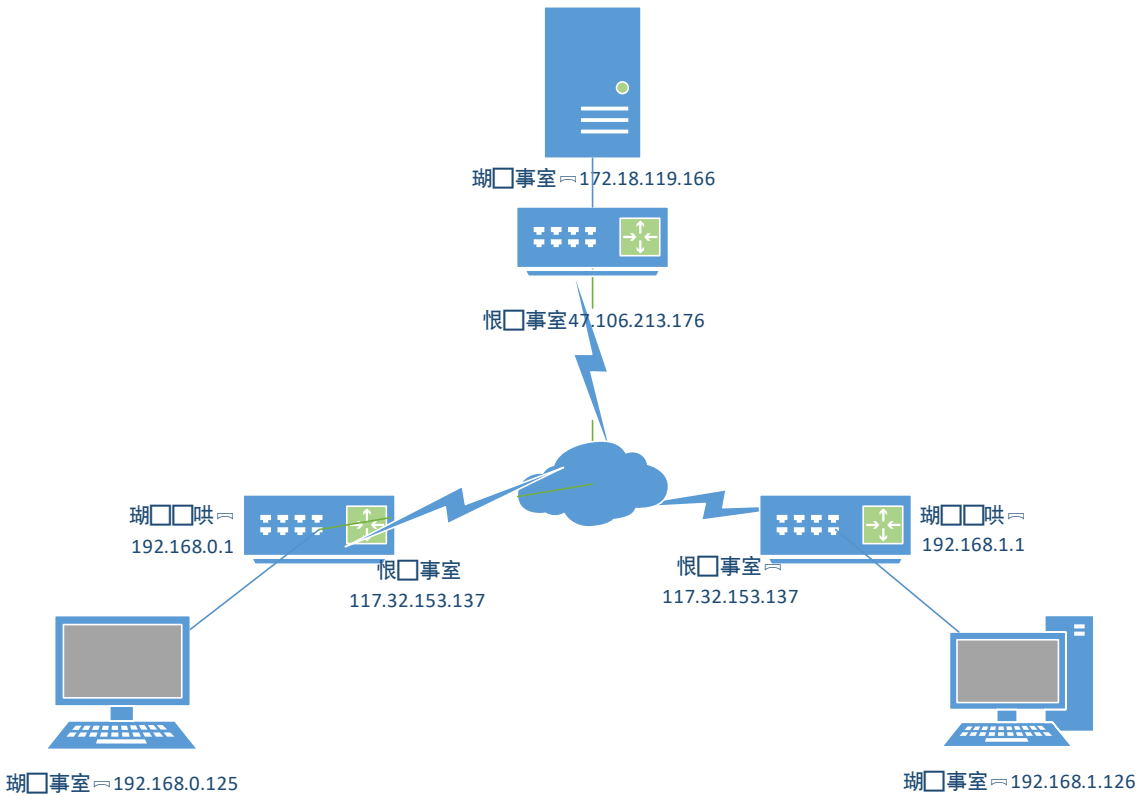
4. 观察 TCP 的流量控制过程，和拥塞控制中的慢启动、快速重传过程。

5. 编写程序实现 TCP 的大数据量的传输。

## 6.3 实验步骤

### 6.3.1 实验环境与分组

每 **2** 名同学一组，以现有校园网络环境及云服务器搭建网络，标注内网、公网地址。




## 6.3.2 TCP 协议分析

步骤 1：PC2 登录到云服务器上，在云服务器上启动 web 服务器（Apache、IIS 等），或者运行 Python 的简单 WEB 服务器。在服务器上放一个约 5MB 的文件。请填表。

| | |
|---|---|
| 云服务器的私有地址 | 172.18.119.166 |
| 云服务器的公网地址 | 47.106.213.176 |
| 文件名 | 5mb-file.bin |
| 文件大小 | 5mb |

步骤 2：在 PC1 和云服务器上启动报文捕获软件，开始截获报文（注意从建立连接开始抓包）。注意：云服务器如果是 Linux 系统，可以用 tcpdump 来截获报文。（完成即可，不要贴图和文字）

步骤 3：在 PC1 上运行**实验 2 编写的 HTTP** 客户端程序，从云服务器上通过 WEB 服务器下载文件。文件传输完成后，停止报文截获。

3.1 修改**实验 2 编写的 HTTP** 客户端程序，把一次下载的时间打印出来（精确到 ms）从云服务器上通过 WEB 服务器下载一个约 5MB 的文件，客户端程序输出文件名和下载花费的时间，请把客户端程序的输出以文字形式贴在下面。

```
python http_client.py
Download file from 47.106.213.176:80/5mb-file.bin? (y/n): y
```

```
Connecting...

[1] DNS Resolve
Hostname: 47.106.213.176
IPV4: 47.106.213.176

[2] TCP Connection
Target: 47.106.213.176:80

[3] HTTP Request
Connection closed.
--------------------------------------------------------------------------------
SUMMARY
--------------------------------------------------------------------------------
DNS Time: 4.565954208374023 ms
TCP Time: 0.579833984375 ms
HTTP Time: 11548.68197441101 ms
Total Time: 11554.149866104126 ms
--------------------------------------------------------------------------------


--------------------------------------------------------------------------------
SAVING FILE
--------------------------------------------------------------------------------
File downloaded successfully as 5mb-file.bin
    File size: 5242880 bytes
```

3.2 文件传输完成后，停止报文截获。通过 HTTP 客户端程序把云服务器上的**报文文件**也下载下来。客户端程序输出文件名和下载花费的时间，请把客户端程序的输出以文字形式贴在下面。

```
    python http_client.py
Download file from 47.106.213.176:80/download_capture.pcap? (y/n):
Connecting...

[1] DNS Resolve
Hostname: 47.106.213.176
IPV4: 47.106.213.176

[2] TCP Connection
Target: 47.106.213.176:80

[3] HTTP Request
Connection closed.
```

```
--------------------------------------------------------------------------------
SUMMARY
--------------------------------------------------------------------------------
DNS Time: 2.4242401123046875 ms
TCP Time: 0.5879402160644531 ms
HTTP Time: 15356.846809387207 ms
Total Time: 15360.146045684814 ms
--------------------------------------------------------------------------------


--------------------------------------------------------------------------------
SAVING FILE
--------------------------------------------------------------------------------
File downloaded successfully as download_capture.pcap.
      File size: 6822677 bytes
```

步骤 4：分析 TCP 协议的 MSS、SACK、Window Scale、Timestamp 等选项。

4.1 查资料，说明 MSS、SACK、Window Scale 等 TCP 选项的用途。

| TCP 选项 | 用途 |
|---|---|
| MSS<br>（**最大报文段大小**） | 用于 TCP 连接建立时，双方通告自身能接收的最大数据段长度。它防止 IP 分片，通常基于链路 MTU 减去 IP/TCP 头计算。较小的 MSS 可减少分片风险，但可能增加头部开销；较大的 MSS 能提升效率。路径 MTU 发现可动态优化此值。 |
| SACK<br>（**选择性确认**） | 允许接收方告知发送方已成功收到的非连续数据块，而非仅确认最大连续序号。当丢包发生时，发送方能只重传真正丢失的片段，避免重传已正确到达的数据，显著提升拥塞控制效率，尤其适用于高丢包率或多丢包的网络环境。 |
| Window Scale<br>（**窗口缩放**） | 解决原始 TCP 窗口字段（16 位，最大 65535 字节）在高速长肥网络（如千兆网、卫星链路）中的带宽延迟积限制。通过握手时交换移位因子，将实际窗口扩展为（窗口值 << 缩放因子），最大可达 1GB，使吞吐量不再受限于小窗口，实现高带宽高效利用。 |

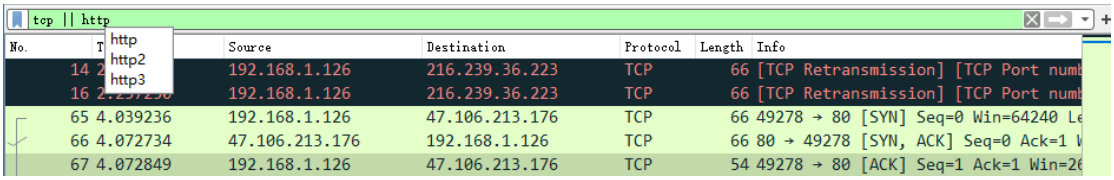
4.2 对比观察 PC1 和云服务器截获的报文，分析 TCP 协议的选项：MSS、SACK、Window Scale 等，结合报文，填表。

| TCP 选项 | 上行报文 | 下行报文 |
|---|---|---|

| | | |
|---|---|---|
| MSS | 1460 | 1460 |
| SACK | 4 | 4 |
| Window Scale | 8 | 7 |
| 实际窗口扩大倍数 | 256 | 128 |

步骤 5：以 TCP 三次握手为准，分析一个 TCP 流的 RTT。

5.1 筛选出**相关**的 TCP 报文，贴图（只显示**过滤器+分组列表，列表中只有三次握手的这 3 个 TCP 分组**）。

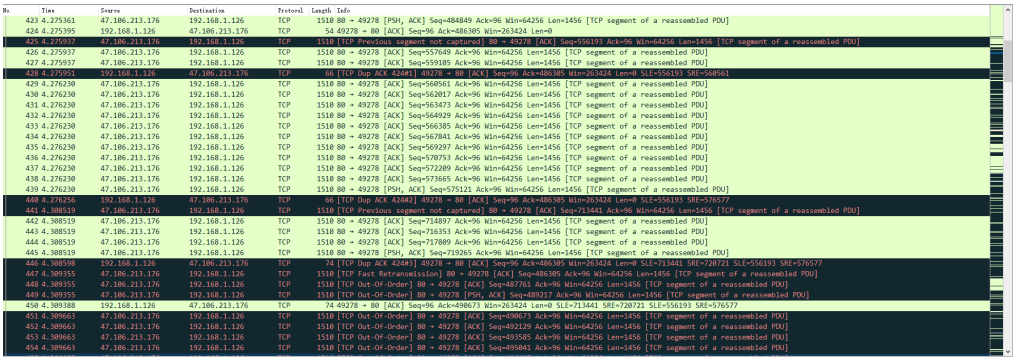



5.2 根据列表中的时间差，计算该 TCP 流的 RTT。（给出算式和结果，精确到 ms）

| |
|---|
| 4.072849000-4.039236000=0.033613000（sec）=33.613ms |

步骤 6：分析前面下载 5MB 文件时的 **TCP 流**数据传送阶段的报文。**务必从建立连接开始抓包**，捕获完整的 TCP 传输过程。

6.1 筛选出**相关**的 TCP 报文，从中找到合适的一段（10-15 个报文），分析其**错误恢复**机制，贴图（限 1 个图，只显示**过滤器+分组列表，列表中只给出填表中需要的 TCP 分组**）。



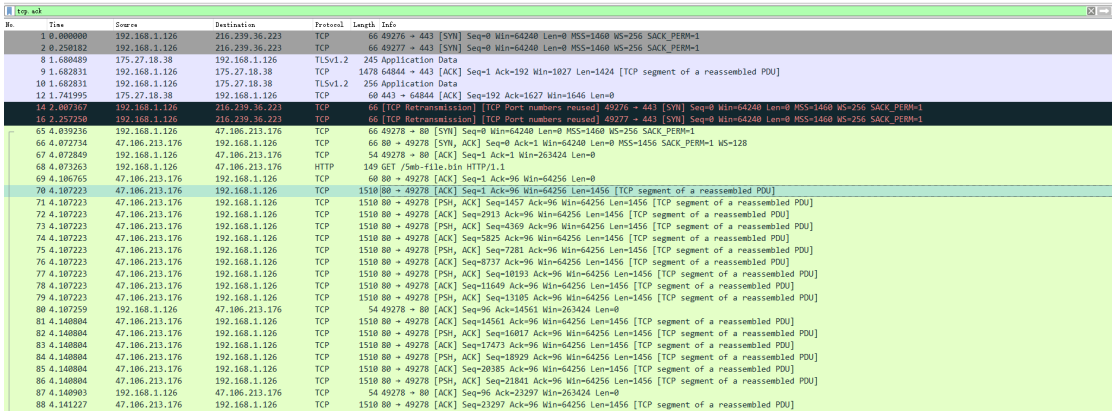

图中有几个错误类型，[TCP Previous segment not captured]为中间有段丢失导致序号不连续，

6.2 分析其**错误恢复**（主要指**累积应答**和**超时重传**）机制并填表（10-15 个报文）。

| 报文号 No. | 序列号 Seq Number | 确认号 Ack Number | 窗口大小 | 数据长度 | 确认到哪条报文(填报文号) |
|---|---|---|---|---|---|

| | | | | | |
|---|---|---|---|---|---|
| 425 | 556193 | 96 | 64256 | 1456 | 68 |
| 428 | 96 | 486305 | 263424 | 0 | 423 |
| 440 | 96 | 486305 | 263424 | 0 | 423 |
| 441 | 713441 | 96 | 64256 | 1456 | 68 |
| 446 | 96 | 486305 | 263424 | 0 | 423 |
| 447 | 486305 | 96 | 64256 | 1456 | （out of order 类型）乱序，无对应报文 |
| 448 | 487761 | 96 | 64256 | 1456 | |
| 449 | 490673 | 96 | 64256 | 1456 | |
| 451 | 492129 | 96 | 64256 | 1456 | |
| 452 | 493585 | 96 | 64256 | 1456 | |

6.3 筛选出**相关**的 TCP 报文，从中找到合适的一段（9-13 个报文），分析其**流量控制**机制，贴图（限 1 个图，只显示**过滤器+分组列表，列表中只给出填表中需要的 TCP 分组**）。



| No. | Time | Source | Destination | Protocol | Length | Info |
|---|---|---|---|---|---|---|
| 1 | 0.000000 | 192.168.1.126 | 216.239.36.223 | TCP | 66 | 49276 → 443 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM=1 |
| 2 | 0.250182 | 192.168.1.126 | 216.239.36.223 | TCP | 66 | 49277 → 443 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM=1 |
| 8 | 1.680489 | 175.27.18.38 | 192.168.1.126 | TLSv1.2 | 245 | Application Data |
| 9 | 1.682831 | 192.168.1.126 | 175.27.18.38 | TCP | 1478 | 64844 → 443 [ACK] Seq=1 Ack=192 Win=1027 Len=1424 [TCP segment of a reassembled PDU] |
| 10 | 1.682831 | 192.168.1.126 | 175.27.18.38 | TLSv1.2 | 256 | Application Data |
| 12 | 1.741995 | 175.27.18.38 | 192.168.1.126 | TCP | 60 | 443 → 64844 [ACK] Seq=192 Ack=1627 Win=1646 Len=0 |
| 14 | 2.007367 | 192.168.1.126 | 216.239.36.223 | TCP | 66 | [TCP Retransmission] [TCP Port numbers reused] 49276 → 443 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM=1 |
| 16 | 2.257250 | 192.168.1.126 | 216.239.36.223 | TCP | 66 | [TCP Retransmission] [TCP Port numbers reused] 49277 → 443 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM=1 |
| 65 | 4.039236 | 192.168.1.126 | 47.106.213.176 | TCP | 66 | 49278 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM=1 |
| 66 | 4.072734 | 47.106.213.176 | 192.168.1.126 | TCP | 66 | 80 → 49278 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1456 SACK_PERM=1 WS=128 |
| 67 | 4.072849 | 192.168.1.126 | 47.106.213.176 | TCP | 54 | 49278 → 80 [ACK] Seq=1 Ack=1 Win=263424 Len=0 |
| 68 | 4.073263 | 192.168.1.126 | 47.106.213.176 | HTTP | 149 | GET /5mb-file.bin HTTP/1.1 |
| 69 | 4.106765 | 47.106.213.176 | 192.168.1.126 | TCP | 60 | 80 → 49278 [ACK] Seq=1 Ack=96 Win=64256 Len=0 |
| 70 | 4.107223 | 47.106.213.176 | 192.168.1.126 | TCP | 1510 | 80 → 49278 [ACK] Seq=1 Ack=96 Win=64256 Len=1456 [TCP segment of a reassembled PDU] |
| 71 | 4.107223 | 47.106.213.176 | 192.168.1.126 | TCP | 1510 | 80 → 49278 [PSH, ACK] Seq=1457 Ack=96 Win=64256 Len=1456 [TCP segment of a reassembled PDU] |
| 72 | 4.107223 | 47.106.213.176 | 192.168.1.126 | TCP | 1510 | 80 → 49278 [ACK] Seq=2913 Ack=96 Win=64256 Len=1456 [TCP segment of a reassembled PDU] |
| 73 | 4.107223 | 47.106.213.176 | 192.168.1.126 | TCP | 1510 | 80 → 49278 [PSH, ACK] Seq=4369 Ack=96 Win=64256 Len=1456 [TCP segment of a reassembled PDU] |
| 74 | 4.107223 | 47.106.213.176 | 192.168.1.126 | TCP | 1510 | 80 → 49278 [ACK] Seq=5825 Ack=96 Win=64256 Len=1456 [TCP segment of a reassembled PDU] |
| 75 | 4.107223 | 47.106.213.176 | 192.168.1.126 | TCP | 1510 | 80 → 49278 [PSH, ACK] Seq=7281 Ack=96 Win=64256 Len=1456 [TCP segment of a reassembled PDU] |
| 76 | 4.107223 | 47.106.213.176 | 192.168.1.126 | TCP | 1510 | 80 → 49278 [ACK] Seq=8737 Ack=96 Win=64256 Len=1456 [TCP segment of a reassembled PDU] |
| 77 | 4.107223 | 47.106.213.176 | 192.168.1.126 | TCP | 1510 | 80 → 49278 [PSH, ACK] Seq=10193 Ack=96 Win=64256 Len=1456 [TCP segment of a reassembled PDU] |
| 78 | 4.107223 | 47.106.213.176 | 192.168.1.126 | TCP | 1510 | 80 → 49278 [ACK] Seq=11649 Ack=96 Win=64256 Len=1456 [TCP segment of a reassembled PDU] |
| 79 | 4.107223 | 47.106.213.176 | 192.168.1.126 | TCP | 1510 | 80 → 49278 [PSH, ACK] Seq=13105 Ack=96 Win=64256 Len=1456 [TCP segment of a reassembled PDU] |
| 80 | 4.107259 | 192.168.1.126 | 47.106.213.176 | TCP | 54 | 49278 → 80 [ACK] Seq=96 Ack=14561 Win=263424 Len=0 |
| 81 | 4.140804 | 47.106.213.176 | 192.168.1.126 | TCP | 1510 | 80 → 49278 [ACK] Seq=14561 Ack=96 Win=64256 Len=1456 [TCP segment of a reassembled PDU] |
| 82 | 4.140804 | 47.106.213.176 | 192.168.1.126 | TCP | 1510 | 80 → 49278 [PSH, ACK] Seq=16017 Ack=96 Win=64256 Len=1456 [TCP segment of a reassembled PDU] |
| 83 | 4.140804 | 47.106.213.176 | 192.168.1.126 | TCP | 1510 | 80 → 49278 [ACK] Seq=17473 Ack=96 Win=64256 Len=1456 [TCP segment of a reassembled PDU] |
| 84 | 4.140804 | 47.106.213.176 | 192.168.1.126 | TCP | 1510 | 80 → 49278 [PSH, ACK] Seq=18929 Ack=96 Win=64256 Len=1456 [TCP segment of a reassembled PDU] |
| 85 | 4.140804 | 47.106.213.176 | 192.168.1.126 | TCP | 1510 | 80 → 49278 [ACK] Seq=20385 Ack=96 Win=64256 Len=1456 [TCP segment of a reassembled PDU] |
| 86 | 4.140804 | 47.106.213.176 | 192.168.1.126 | TCP | 1510 | 80 → 49278 [PSH, ACK] Seq=21841 Ack=96 Win=64256 Len=1456 [TCP segment of a reassembled PDU] |
| 87 | 4.140903 | 192.168.1.126 | 47.106.213.176 | TCP | 54 | 49278 → 80 [ACK] Seq=96 Ack=23297 Win=263424 Len=0 |
| 88 | 4.141227 | 47.106.213.176 | 192.168.1.126 | TCP | 1510 | 80 → 49278 [ACK] Seq=23297 Ack=96 Win=64256 Len=1456 [TCP segment of a reassembled PDU] |

全程正常工作（win 值除了一开始 tcp 握手之外，基本上没有任何变化），窗口大小稳定，说明客户端接收缓冲区充足，服务器可以全速传输

6.4 分析其**流量控制**机制并填表。

| 报文号 No. | 序列号 Seq Number | 确认号 Ack Number | 窗口大小 | 数据长度 | 确认到哪条报文 (填报文号) |
|---|---|---|---|---|---|
| 65 | 0 | | 64240 | 0 | - |
| 66 | 0 | 1 | 64240 | 0 | - |
| 67 | 1 | 1 | 263424 | 0 | - |

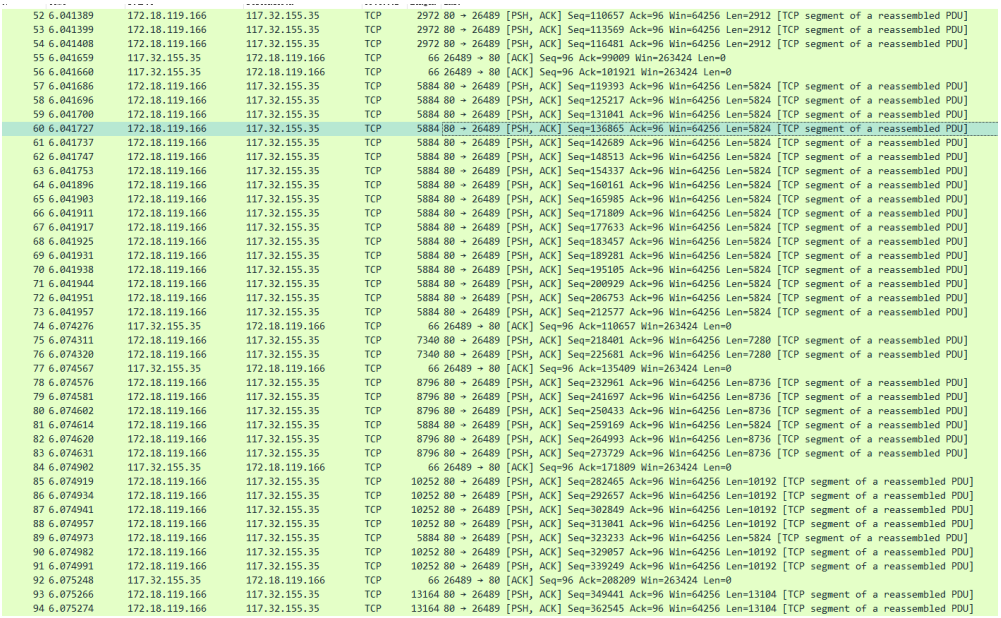
| | | | | | |
|---|---|---|---|---|---|
| 69 | 1 | 96 | 64256 | 0 | 68 |
| 70 | 1 | 96 | 64256 | 1456 | 68 |
| 71 | 1457 | 96 | 64256 | 1456 | 68 |
| 72 | 2913 | 96 | 64256 | 1456 | 68 |
| 73 | 4369 | 96 | 64256 | 1456 | 68 |
| 74 | 5825 | 96 | 64256 | 1456 | 68 |
| 75 | 7281 | 96 | 64256 | 1456 | 68 |

6.5 对 6.3 的分组，给出文字解释流量控制的过程。

TCP 通过接收窗口动态反馈实现流量控制的过程：接收方实时告知发送方“传输权限”，发送方依此调整速率，最终实现高效且无溢出的传输。

步骤 7、分析 TCP 流的慢启动、快速重传拥塞控过程。（如果拥塞控制的相关过程不明显，请设计合适的方法再次测试。）观察服务器的初始 cwnd。

7.1 分析 TCP 流的**慢启动**过程，贴图展示（限 1 个图，只显示**分组列表，列表中只给出**慢启动过程**的 TCP 分组**）。



| | | | | | | |
|---|---|---|---|---|---|---|
| 52 | 6.041389 | 172.18.119.166 | 117.32.155.35 | TCP | 2972 | 80 → 26489 [PSH, ACK] Seq=110657 Ack=96 Win=64256 Len=2912 [TCP segment of a reassembled PDU] |
| 53 | 6.041399 | 172.18.119.166 | 117.32.155.35 | TCP | 2972 | 80 → 26489 [PSH, ACK] Seq=113569 Ack=96 Win=64256 Len=2912 [TCP segment of a reassembled PDU] |
| 54 | 6.041408 | 172.18.119.166 | 117.32.155.35 | TCP | 2972 | 80 → 26489 [PSH, ACK] Seq=116481 Ack=96 Win=64256 Len=2912 [TCP segment of a reassembled PDU] |
| 55 | 6.041659 | 117.32.155.35 | 172.18.119.166 | TCP | 66 | 26489 → 80 [ACK] Seq=96 Ack=99009 Win=263424 Len=0 |
| 56 | 6.041660 | 117.32.155.35 | 172.18.119.166 | TCP | 66 | 26489 → 80 [ACK] Seq=96 Ack=101921 Win=263424 Len=0 |
| 57 | 6.041686 | 172.18.119.166 | 117.32.155.35 | TCP | 5884 | 80 → 26489 [PSH, ACK] Seq=119393 Ack=96 Win=64256 Len=5824 [TCP segment of a reassembled PDU] |
| 58 | 6.041696 | 172.18.119.166 | 117.32.155.35 | TCP | 5884 | 80 → 26489 [PSH, ACK] Seq=125217 Ack=96 Win=64256 Len=5824 [TCP segment of a reassembled PDU] |
| 59 | 6.041700 | 172.18.119.166 | 117.32.155.35 | TCP | 5884 | 80 → 26489 [PSH, ACK] Seq=131041 Ack=96 Win=64256 Len=5824 [TCP segment of a reassembled PDU] |
| 60 | 6.041727 | 172.18.119.166 | 117.32.155.35 | TCP | 5884 | 80 → 26489 [PSH, ACK] Seq=136865 Ack=96 Win=64256 Len=5824 [TCP segment of a reassembled PDU] |
| 61 | 6.041737 | 172.18.119.166 | 117.32.155.35 | TCP | 5884 | 80 → 26489 [PSH, ACK] Seq=142689 Ack=96 Win=64256 Len=5824 [TCP segment of a reassembled PDU] |
| 62 | 6.041747 | 172.18.119.166 | 117.32.155.35 | TCP | 5884 | 80 → 26489 [PSH, ACK] Seq=148513 Ack=96 Win=64256 Len=5824 [TCP segment of a reassembled PDU] |
| 63 | 6.041753 | 172.18.119.166 | 117.32.155.35 | TCP | 5884 | 80 → 26489 [PSH, ACK] Seq=154337 Ack=96 Win=64256 Len=5824 [TCP segment of a reassembled PDU] |
| 64 | 6.041896 | 172.18.119.166 | 117.32.155.35 | TCP | 5884 | 80 → 26489 [PSH, ACK] Seq=160161 Ack=96 Win=64256 Len=5824 [TCP segment of a reassembled PDU] |
| 65 | 6.041903 | 172.18.119.166 | 117.32.155.35 | TCP | 5884 | 80 → 26489 [PSH, ACK] Seq=165985 Ack=96 Win=64256 Len=5824 [TCP segment of a reassembled PDU] |
| 66 | 6.041911 | 172.18.119.166 | 117.32.155.35 | TCP | 5884 | 80 → 26489 [PSH, ACK] Seq=171809 Ack=96 Win=64256 Len=5824 [TCP segment of a reassembled PDU] |
| 67 | 6.041917 | 172.18.119.166 | 117.32.155.35 | TCP | 5884 | 80 → 26489 [PSH, ACK] Seq=177633 Ack=96 Win=64256 Len=5824 [TCP segment of a reassembled PDU] |
| 68 | 6.041925 | 172.18.119.166 | 117.32.155.35 | TCP | 5884 | 80 → 26489 [PSH, ACK] Seq=183457 Ack=96 Win=64256 Len=5824 [TCP segment of a reassembled PDU] |
| 69 | 6.041931 | 172.18.119.166 | 117.32.155.35 | TCP | 5884 | 80 → 26489 [PSH, ACK] Seq=189281 Ack=96 Win=64256 Len=5824 [TCP segment of a reassembled PDU] |
| 70 | 6.041938 | 172.18.119.166 | 117.32.155.35 | TCP | 5884 | 80 → 26489 [PSH, ACK] Seq=195105 Ack=96 Win=64256 Len=5824 [TCP segment of a reassembled PDU] |
| 71 | 6.041944 | 172.18.119.166 | 117.32.155.35 | TCP | 5884 | 80 → 26489 [PSH, ACK] Seq=200929 Ack=96 Win=64256 Len=5824 [TCP segment of a reassembled PDU] |
| 72 | 6.041951 | 172.18.119.166 | 117.32.155.35 | TCP | 5884 | 80 → 26489 [PSH, ACK] Seq=206753 Ack=96 Win=64256 Len=5824 [TCP segment of a reassembled PDU] |
| 73 | 6.041957 | 172.18.119.166 | 117.32.155.35 | TCP | 5884 | 80 → 26489 [PSH, ACK] Seq=212577 Ack=96 Win=64256 Len=5824 [TCP segment of a reassembled PDU] |
| 74 | 6.074276 | 117.32.155.35 | 172.18.119.166 | TCP | 66 | 26489 → 80 [ACK] Seq=96 Ack=110657 Win=263424 Len=0 |
| 75 | 6.074311 | 172.18.119.166 | 117.32.155.35 | TCP | 7340 | 80 → 26489 [PSH, ACK] Seq=218401 Ack=96 Win=64256 Len=7280 [TCP segment of a reassembled PDU] |
| 76 | 6.074320 | 172.18.119.166 | 117.32.155.35 | TCP | 7340 | 80 → 26489 [PSH, ACK] Seq=225681 Ack=96 Win=64256 Len=7280 [TCP segment of a reassembled PDU] |
| 77 | 6.074567 | 117.32.155.35 | 172.18.119.166 | TCP | 66 | 26489 → 80 [ACK] Seq=96 Ack=135409 Win=263424 Len=0 |
| 78 | 6.074576 | 172.18.119.166 | 117.32.155.35 | TCP | 8796 | 80 → 26489 [PSH, ACK] Seq=232961 Ack=96 Win=64256 Len=8736 [TCP segment of a reassembled PDU] |
| 79 | 6.074581 | 172.18.119.166 | 117.32.155.35 | TCP | 8796 | 80 → 26489 [PSH, ACK] Seq=241697 Ack=96 Win=64256 Len=8736 [TCP segment of a reassembled PDU] |
| 80 | 6.074602 | 172.18.119.166 | 117.32.155.35 | TCP | 8796 | 80 → 26489 [PSH, ACK] Seq=250433 Ack=96 Win=64256 Len=8736 [TCP segment of a reassembled PDU] |
| 81 | 6.074614 | 172.18.119.166 | 117.32.155.35 | TCP | 5884 | 80 → 26489 [PSH, ACK] Seq=259169 Ack=96 Win=64256 Len=5824 [TCP segment of a reassembled PDU] |
| 82 | 6.074620 | 172.18.119.166 | 117.32.155.35 | TCP | 8796 | 80 → 26489 [PSH, ACK] Seq=264993 Ack=96 Win=64256 Len=8736 [TCP segment of a reassembled PDU] |
| 83 | 6.074631 | 172.18.119.166 | 117.32.155.35 | TCP | 8796 | 80 → 26489 [PSH, ACK] Seq=273729 Ack=96 Win=64256 Len=8736 [TCP segment of a reassembled PDU] |
| 84 | 6.074902 | 117.32.155.35 | 172.18.119.166 | TCP | 66 | 26489 → 80 [ACK] Seq=96 Ack=171809 Win=263424 Len=0 |
| 85 | 6.074919 | 172.18.119.166 | 117.32.155.35 | TCP | 10252 | 80 → 26489 [PSH, ACK] Seq=282465 Ack=96 Win=64256 Len=10192 [TCP segment of a reassembled PDU] |
| 86 | 6.074934 | 172.18.119.166 | 117.32.155.35 | TCP | 10252 | 80 → 26489 [PSH, ACK] Seq=292657 Ack=96 Win=64256 Len=10192 [TCP segment of a reassembled PDU] |
| 87 | 6.074941 | 172.18.119.166 | 117.32.155.35 | TCP | 10252 | 80 → 26489 [PSH, ACK] Seq=302849 Ack=96 Win=64256 Len=10192 [TCP segment of a reassembled PDU] |
| 88 | 6.074957 | 172.18.119.166 | 117.32.155.35 | TCP | 10252 | 80 → 26489 [PSH, ACK] Seq=313041 Ack=96 Win=64256 Len=10192 [TCP segment of a reassembled PDU] |
| 89 | 6.074973 | 172.18.119.166 | 117.32.155.35 | TCP | 5884 | 80 → 26489 [PSH, ACK] Seq=323233 Ack=96 Win=64256 Len=5824 [TCP segment of a reassembled PDU] |
| 90 | 6.074982 | 172.18.119.166 | 117.32.155.35 | TCP | 10252 | 80 → 26489 [PSH, ACK] Seq=329057 Ack=96 Win=64256 Len=10192 [TCP segment of a reassembled PDU] |
| 91 | 6.074991 | 172.18.119.166 | 117.32.155.35 | TCP | 10252 | 80 → 26489 [PSH, ACK] Seq=339249 Ack=96 Win=64256 Len=10192 [TCP segment of a reassembled PDU] |
| 92 | 6.075248 | 117.32.155.35 | 172.18.119.166 | TCP | 66 | 26489 → 80 [ACK] Seq=96 Ack=208209 Win=263424 Len=0 |
| 93 | 6.075266 | 172.18.119.166 | 117.32.155.35 | TCP | 13164 | 80 → 26489 [PSH, ACK] Seq=349441 Ack=96 Win=64256 Len=13104 [TCP segment of a reassembled PDU] |
| 94 | 6.075274 | 172.18.119.166 | 117.32.155.35 | TCP | 13164 | 80 → 26489 [PSH, ACK] Seq=362545 Ack=96 Win=64256 Len=13104 [TCP segment of a reassembled PDU] |

7.2 对这个图给出文字解释其**慢启动**过程。

三次握手后，发送方以 MSS 为单位发送数据，序列号阶梯递增；每收到 ACK 后拥塞窗口 cwnd 指数增长，报文无异常标记，是连接初期慢启动的典型表现。

7.3 分析 TCP 流的**快速重传**过程，贴图展示（限 1 个图，只显示**分组列表，**

**列表中只给出**快速重传过程**的 TCP 分组**）。

| | | | | | |
|---|---|---|---|---|---|
| 434 | 4.276230 | 47.106.213.176 | 192.168.1.126 | TCP | 1510 80 → 49278 [ACK] Seq=567841 Ack=96 Win=64256 Len=1456 [TCP segment of a reassembled PDU] |
| 435 | 4.276230 | 47.106.213.176 | 192.168.1.126 | TCP | 1510 80 → 49278 [ACK] Seq=569297 Ack=96 Win=64256 Len=1456 [TCP segment of a reassembled PDU] |
| 436 | 4.276230 | 47.106.213.176 | 192.168.1.126 | TCP | 1510 80 → 49278 [ACK] Seq=570753 Ack=96 Win=64256 Len=1456 [TCP segment of a reassembled PDU] |
| 437 | 4.276230 | 47.106.213.176 | 192.168.1.126 | TCP | 1510 80 → 49278 [ACK] Seq=572209 Ack=96 Win=64256 Len=1456 [TCP segment of a reassembled PDU] |
| 438 | 4.276230 | 47.106.213.176 | 192.168.1.126 | TCP | 1510 80 → 49278 [ACK] Seq=573665 Ack=96 Win=64256 Len=1456 [TCP segment of a reassembled PDU] |
| 439 | 4.276230 | 47.106.213.176 | 192.168.1.126 | TCP | 1510 80 → 49278 [PSH, ACK] Seq=575121 Ack=96 Win=64256 Len=1456 [TCP segment of a reassembled PDU] |
| 440 | 4.276256 | 192.168.1.126 | 47.106.213.176 | TCP | 66 [TCP Dup ACK 424#2] 49278 → 80 [ACK] Seq=96 Ack=486305 Win=263424 Len=0 SLE=556193 SRE=576577 |
| 441 | 4.308519 | 47.106.213.176 | 192.168.1.126 | TCP | 1510 [TCP Previous segment not captured] 80 → 49278 [ACK] Seq=713441 Ack=96 Win=64256 Len=1456 [TCP segment of a reassembled PDU] |
| 442 | 4.308519 | 47.106.213.176 | 192.168.1.126 | TCP | 1510 80 → 49278 [ACK] Seq=714897 Ack=96 Win=64256 Len=1456 [TCP segment of a reassembled PDU] |
| 443 | 4.308519 | 47.106.213.176 | 192.168.1.126 | TCP | 1510 80 → 49278 [ACK] Seq=716353 Ack=96 Win=64256 Len=1456 [TCP segment of a reassembled PDU] |
| 444 | 4.308519 | 47.106.213.176 | 192.168.1.126 | TCP | 1510 80 → 49278 [ACK] Seq=717809 Ack=96 Win=64256 Len=1456 [TCP segment of a reassembled PDU] |
| 445 | 4.308519 | 47.106.213.176 | 192.168.1.126 | TCP | 1510 80 → 49278 [PSH, ACK] Seq=719265 Ack=96 Win=64256 Len=1456 [TCP segment of a reassembled PDU] |
| 446 | 4.308598 | 192.168.1.126 | 47.106.213.176 | TCP | 74 [TCP Dup ACK 424#3] 49278 → 80 [ACK] Seq=96 Ack=486305 Win=263424 Len=0 SLE=713441 SRE=720721 SLE=556193 SRE=576577 |
| 447 | 4.309355 | 47.106.213.176 | 192.168.1.126 | TCP | 1510 [TCP Fast Retransmission] 80 → 49278 [ACK] Seq=486305 Ack=96 Win=64256 Len=1456 [TCP segment of a reassembled PDU] |
| 448 | 4.309355 | 47.106.213.176 | 192.168.1.126 | TCP | 1510 [TCP Out-Of-Order] 80 → 49278 [ACK] Seq=487761 Ack=96 Win=64256 Len=1456 [TCP segment of a reassembled PDU] |
| 449 | 4.309355 | 47.106.213.176 | 192.168.1.126 | TCP | 1510 [TCP Out-Of-Order] 80 → 49278 [PSH, ACK] Seq=489217 Ack=96 Win=64256 Len=1456 [TCP segment of a reassembled PDU] |
| 450 | 4.309388 | 192.168.1.126 | 47.106.213.176 | TCP | 74 49278 → 80 [ACK] Seq=96 Ack=490673 Win=263424 Len=0 SLE=713441 SRE=720721 SLE=556193 SRE=576577 |

7.4 对这个图给出文字解释 TCP 流的**快速重传**过程。

| |
|---|
| 客户端因丢包连续发 Dup ACK，服务器收到 3 个重复 ACK 后，触发 Fast Retransmission，立即重传丢失的 Seq=486305 段，无需等超时，后续报文因乱序被标记，最终完成丢包恢复，保障传输。 |

7.5 观察服务器的初始 cwnd 是多少（个 MSS）。

| | |
|---|---|
| 服务器的 TCP 初始 cwn（个 MSS） | 10 个 |

## 6.3.3 TCP 文件传输的编程

步骤 8：设计基于 TCP 协议的文件传输协议，编写文件传输的服务器程序。该服务器程序能够在客户端请求文件时，打开对应的文件，将文件内容通过 TCP 协议传输给客户端。给出代码（文字）。

```
import socket


    def get_private_server_ip() -> str:
        with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as
sock:
            try:
                sock.connect(("8.8.8.8", 80))
                return sock.getsockname()[0]
            except OSError:
                return "127.0.0.1"


    def start_server(host="0.0.0.0", port=65432):
        with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as
server_socket:
            server_socket.setsockopt(socket.SOL_SOCKET,
```

```
socket.SO_REUSEADDR, 1)
            server_socket.bind((host, port))
            server_socket.listen()
            print(f"Server listening on {host}:{port}",
flush=True)
            conn, addr = server_socket.accept()
            with conn:
                client_ip, client_port = addr
                server_private_ip = get_private_server_ip()
                print(f"Client public IP: {client_ip}",
flush=True)
                print(f"Server private IP: {server_private_ip}",
flush=True)
                print(f"Connected by {addr}", flush=True)

                data = conn.recv(1024)
                if not data:
                    print(
                        "No filename received from client, closing
connection.", flush=True
                    )
                    return

                file_name = data.decode().strip()
                if not file_name:
                    print("Received empty filename, closing
connection.", flush=True)
                    return

                print(f"Requested file: {file_name}", flush=True)
                try:
                    with open(file_name, "rb") as f:
                        print(f"Starting transfer of
{file_name}.", flush=True)
                        while chunk := f.read(1024):
                            try:
                                conn.sendall(chunk)
                            except (BrokenPipeError,
ConnectionResetError):
                                print(
```

```
                                "Client disconnected before transfer finished.",
                                flush=True,
                            )
                            return
                    try:
                        conn.shutdown(socket.SHUT_WR)
                    except OSError:
                        pass
                    print("File transfer complete. Connection closed.", flush=True)
                except FileNotFoundError:
                    print(f"File not found: {file_name}", flush=True)
                    try:
                        conn.sendall(b"File not found")
                    except (BrokenPipeError, ConnectionResetError):
                        pass
                    return


    if __name__ == "__main__":
        start_server()
```

步骤 9：编写文件传输客户端程序。该客户端程序能够连接到步骤 8 的服务器程序，根据用户输入请求一个文件，并从服务器接收文件内容，保存为一个文件。给出代码（文字）。

```
import socket


    def request_file(file_name, host="47.106.213.176", port=65432):
        with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as client_socket:
            client_socket.connect((host, port))
```

```
        client_socket.sendall(file_name.encode())
        first_chunk = client_socket.recv(1024)
        if first_chunk == b"File not found":
            print("File not found on server.")
            return

        output_path = f"received_{file_name}"
        with open(output_path, "wb") as f:
            f.write(first_chunk)
            while data := client_socket.recv(1024):
                f.write(data)

        print(f"File {file_name} received successfully.")


if __name__ == "__main__":
    request_file("file2.txt")
```

步骤 10：将服务器部署在云服务器上，在本地机器上运行客户端程序，测试传输两个文件（500KB 左右的大小），贴图。

```
~/Lo/G/XJTU-Labs/Computer-N/lab-6 | on main !5 ?8
python3 file_transfer_client.py
File file1.txt received successfully.

~/Lo/G/XJTU-Labs/Computer-N/lab-6 | on main !5 ?8
python3 file_transfer_client.py
File file2.txt received successfully.
```

步骤 11：修改 TCP 服务器程序，在云服务器上运行，输出客户端的公网地址（及端口）和云服务器的私网地址。给出代码（文字）。

```
import socket


def get_private_server_ip() -> str:
    with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as
sock:
        try:
            sock.connect(("8.8.8.8", 80))
```

```
            return sock.getsockname()[0]
        except OSError:
            return "127.0.0.1"


    def start_server(host="0.0.0.0", port=65432):
        with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as
server_socket:
            server_socket.setsockopt(socket.SOL_SOCKET,
socket.SO_REUSEADDR, 1)
            server_socket.bind((host, port))
            server_socket.listen()
            print(f"Server    listening    on    {host}:{port}",
flush=True)
            conn, addr = server_socket.accept()
            with conn:
                client_ip, client_port = addr
                server_private_ip = get_private_server_ip()
                print(f"Client    public    IP:    {client_ip}",
flush=True)
                print(f"Server private IP: {server_private_ip}",
flush=True)
                print(f"Connected by {addr}", flush=True)

                data = conn.recv(1024)
                if not data:
                    print(
                        "No filename received from client, closing
connection.", flush=True
                    )
                    return

                file_name = data.decode().strip()
                if not file_name:
                    print("Received   empty   filename,   closing
connection.", flush=True)
                    return

                print(f"Requested file: {file_name}", flush=True)
                try:
```

```
                    with open(file_name, "rb") as f:
                        print(f"Starting transfer of {file_name}.", flush=True)
                        while chunk := f.read(1024):
                            try:
                                conn.sendall(chunk)
                            except (BrokenPipeError, ConnectionResetError):
                                print(
                                    "Client disconnected before transfer finished.",
                                    flush=True,
                                )
                                return
                        try:
                            conn.shutdown(socket.SHUT_WR)
                        except OSError:
                            pass
                        print("File transfer complete. Connection closed.", flush=True)
                except FileNotFoundError:
                    print(f"File not found: {file_name}", flush=True)
                    try:
                        conn.sendall(b"File not found")
                    except (BrokenPipeError, ConnectionResetError):
                        pass
                    return


    if __name__ == "__main__":
        start_server()
```

## 6.4 互动讨论主题

步骤 12：请答题。

12.1 TCP 的拥塞控制是哪一方（接收、发送）来主导的？拥塞控制的目的是什么？

| |
|---|
| 由发送方主导，为了防止发送方发送速率过高，导致路由器出现缓冲区溢出，丢包，延迟激增，为保证整个网络的稳定与公平，避免全局拥塞 |

12.2 TCP 的流量控制是哪一方（接收、发送）来主导的？流量控制的目的是什么？

| |
|---|
| 由接收方主导，为防止发送方发送速率过高导致接收方接受缓冲区溢出，丢包 |

12.3 说明 TCP 与应用层的关系，包括数据传递、缓冲区的管理、可靠性的保证等。

| |
|---|
| TCP 为应用层提供可靠字节流服务：应用层将数据写入 TCP 发送缓冲区，TCP 分段封装、通过拥塞 / 流量控制发送；接收端 TCP 在接收缓冲区排序重组，交付应用。TCP 全权保障数据可靠、有序、不丢包，让应用层无需处理传输细节。 |